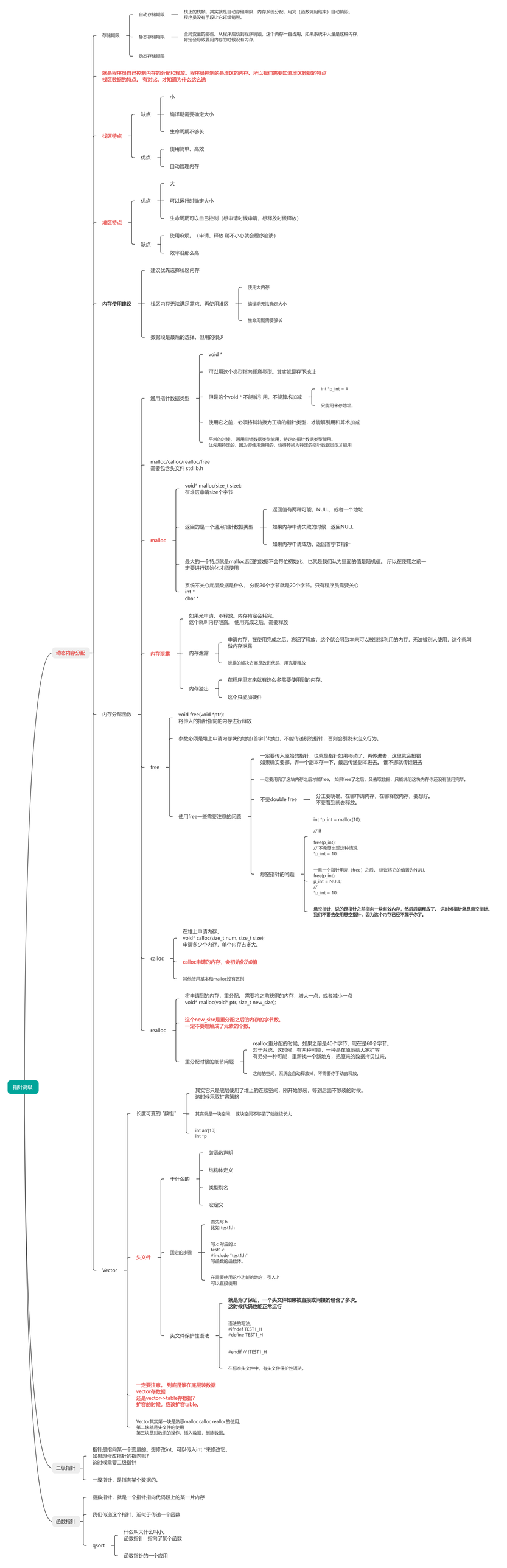
指针高级
动态内存分配
存储期限
自动存储期限
线上的栈帧，其实就是自动存储期限，内存系统分配，用完（函数调用结束）自动销毁。
程序员没有手段让它延缓销毁。
静态存储期限
全局变量的那些，从程序启动到程序销毁，这个内存一直占用。如果系统中大量是这种内存，
肯定会导致要用内存的时候没有内存。
动态存储期限
就是程序员自己控制内存的分配和释放。程序员控制的是堆区的内存。所以我们需要知道堆区数据的特点
栈区数据的特点。 有对比，才知道为什么这么选
栈区特点
缺点
小
编译期需要确定大小
生命周期不够长
优点
使用简单、高效
自动管理内存
堆区特点
优点
大
可以运行时确定大小
生命周期可以自己控制（想申请时候申请，想释放时候释放）
缺点
使用麻烦。（申请、释放 稍不小心就会程序崩溃）
效率没那么高
内存使用建议
建议优先选择栈区内存
栈区内存无法满足需求，再使用堆区
使用大内存
编译期无法确定大小
生命周期需要够长
数据段是最后的选择，但用的很少
内存分配函数
通用指针数据类型
void *
可以用这个类型指向任意类型。其实就是存下地址
但是这个void * 不能解引用，不能算术加减
int *p_int = #
只能用来存地址。
使用它之前，必须将其转换为正确的指针类型，才能解引用和算术加减
平常的时候，通用指针数据类型能用，特定的指针数据类型能用。
优先用特定的，因为即使用通用的，也得转换为特定的指针数据类型才能用
malloc/calloc/realloc/free
需要包含头文件 stdlib.h
malloc
void* malloc(size_t size);
在堆区申请size个字节
返回的是一个通用指针数据类型
返回值有两种可能，NULL，或者一个地址
如果内存申请失败的时候，返回NULL
如果内存申请成功，返回首字节指针
最大的一个特点就是malloc返回的数据不会帮忙初始化，也就是我们认为里面的值是随机值。 所以在使用之前一
定要进行初始化才能使用
系统不关心底层数据是什么， 分配20个字节就是20个字节。只有程序员需要关心
int *
char *
内存泄露
如果光申请，不释放。内存肯定会耗完。
这个就叫内存泄露。 使用完成之后，需要释放
内存泄露
申请内存，在使用完成之后。忘记了释放，这个就会导致本来可以被继续利用的内存，无法被别人使用，这个就叫
做内存泄露
泄露的解决方案是改进代码，用完要释放
内存溢出
在程序里本来就有这么多需要使用到的内存。
这个只能加硬件
free
void free(void *ptr);
将传入的指针指向的内存进行释放
参数必须是堆上申请内存块的地址(首字节地址)，不能传递别的指针，否则会引发未定义行为。
使用free一些需要注意的问题
一定要传入原始的指针，也就是指针如果移动了，再传进去，这里就会报错
如果确实要挪，弄一个副本存一下。最后传递副本进去， 谁不挪就传谁进去
一定要用完了这块内存之后才能free， 如果free了之后，又去取数据，只能说明这块内存你还没有使用完毕。
不要double free
分工要明确。在哪申请内存，在哪释放内存，要想好。
不要看到就去释放。
悬空指针的问题
int *p_int = malloc(10);
// if
free(p_int);
// 不希望出现这种情况
*p_int = 10;
一旦一个指针用完（free）之后， 建议将它的值置为NULL
free(p_int);
p_int = NULL;
//
*p_int = 10;
悬空指针，说的是指针之前指向一块有效内存，然后后期释放了， 这时候指针就是悬空指针。
我们不要去使用悬空指针，因为这个内存已经不属于你了。
calloc
在堆上申请内存，
void* calloc(size_t num, size_t size);
申请多少个内存，单个内存占多大。
calloc申请的内存，会初始化为0值
其他使用基本和malloc没有区别
realloc
将申请到的内存，重分配， 需要将之前获得的内存，增大一点，或者减小一点
void* realloc(void* ptr, size_t new_size);
这个new_size是重分配之后的内存的字节数。
一定不要理解成了元素的个数。
重分配时候的细节问题
realloc重分配的时候。如果之前是40个字节，现在是60个字节。
对于系统，这时候， 有两种可能，一种是在原地给大家扩容
有另外一种可能，重新找一个新地方，把原来的数据拷贝过来。
之前的空间，系统会自动释放掉，不需要你手动去释放。
Vector
长度可变的“数组”
其实它只是底层使用了堆上的连续空间，刚开始够装，等到后面不够装的时候。
这时候采取扩容策略
其实就是一块空间， 这块空间不够装了就继续长大
int arr[10]
int *p
头文件
干什么的
装函数声明
结构体定义
类型别名
宏定义
固定的步骤
首先写.h
比如 test1.h
写.c 对应的.c
test1.c
#include "test1.h"
写函数的函数体。
在需要使用这个功能的地方，引入.h
可以直接使用
头文件保护性语法
就是为了保证，一个头文件如果被直接或间接的包含了多次。
这时候代码也能正常运行
语法的写法。
#ifndef TEST1_H
#define TEST1_H
#endif // !TEST1_H
在标准头文件中，有头文件保护性语法。
一定要注意。 到底是谁在底层装数据
vector存数据
还是vector->table存数据?
扩容的时候，应该扩容table。
Vector其实第一块是熟悉malloc calloc realloc的使用。
第二块就是头文件的使用
第三块是对数组的操作，插入数据，删除数据。
二级指针
指针是指向某一个变量的。想修改int，可以传入int *来修改它。
如果想修改指针的指向呢?
这时候需要二级指针
一级指针，是指向某个数据的。
函数指针
函数指针，就是一个指针指向代码段上的某一片内存
我们传递这个指针，近似于传递一个函数
qsort
什么叫大什么叫小。
函数指针 指向了某个函数
函数指针的一个应用